



QuESO Official Manual

Release v0.0

Sarah Riley

May 07, 2026

CONTENTS:

1	Project Contributors	3
2	Introduction to k-means clustering of spectra	5
3	Installation	7
4	API Reference	9
4.1	writer.py	9
4.2	aux.py	9
4.3	base.py	9
4.4	approach.py	10
4.5	logg.py	11
4.6	aia.py	11
4.7	style.py	11
4.8	multiline.py	11
4.9	products.py	11
4.10	loader.py	11
4.11	base.py	12
4.12	base.py	12

 Warning

This documentation is under active development.

Quantifying the Evolution of Spectra with Optimization (QuESO; `queso-cluster`) designed around applying k-means clustering to spectroscopic observations. The primary objective of this package is to develop and streamline the clustering of spectroscopic observations of solar flares to study the evolution of line profiles.

At present, we only support observations from the Visible Spectropolarimeter (ViSP) on the Daniel In-oyue Solar Telescope (DKIST). We do have plans to expand this to the Fast Imaging Solar Spectrograph (FISS) on the Goode Solar Telescope (GST) and the Interface Region Imaging Spectrograph (IRIS).

PROJECT CONTRIBUTORS

Sarah Riley
Lead Developer https://sriley.dev

INTRODUCTION TO K-MEANS CLUSTERING OF SPECTRA

The k-means clustering algorithm is the most used clustering algorithm for clustering spectral profiles.

INSTALLATION

You can install the latest version of this package from GitHub by

```
` pip install git+https://github.com/SharkSorceress/queso-cluster.git `
```

Once we have a proper version, we plan to submit our package to PyPi as queso-cluster.

API REFERENCE

4.1 writer.py

exportFITS(*spectralDataset*, *labelLine*, *fname*)

4.2 aux.py

_gen_dataID(*Input*)

pick_jth_label(*labelLst*, *j*)

density_2channel(*x*, *y*, *dy*, *xsize*, *top*, *bottom*)

density_hist2d(*data*, *dy*, *top*, *bottom*)

close_factors(*number*)

almost_factors(*number*)

common_elements(*ar1*, *ar2*, *ar3*)

4.3 base.py

normZ(*dataSquare*)

normMaximum(*dataSquare*)

normContinuum(*dataSquare*, *continuumIndx*)

concatSpectra(*dataSquareLst*)

numba_histogram(*a*, *bins*, *lim*)

rotateArray(*image*, *turns*)

get_bin_edges(*bins*, *lim*)

compute_bin(*x*, *bin_edges*)

np_gradient(*f*)

minimize(*data*, *decisions*, *size*)

maximize(*data, decisions, size*)
np_all_axis0(*x*)
np_all_axis1(*x*)
similarityMetric(*x, y, type='dist', ref=0*)
curvature(*y*)
startMax(*data, k, decisions*)
startPlusPlus(*data, k, decisions*)
_calcMoment(*waveAxis, ii, jj, lineCore, dataCube, order, ref, counter=0*)
_calcFeatureDensity(*data, converge, zindx, func1*)
_calcOptimization(*k, data, decision, threshold*)
_calcElbowEntry(*data, labels*)
_calcVarianceScore(*data*)
_criteriaInertiaScore(*score*)
_calcInertiaScore(*dataSquare, labelLine*)
_criteriaSilhouetteScore(*score*)
_calcSilhouetteScore(*dataSquare, labelLine*)
_calcSingleSilhouetteScore(*data, labels, lab*)
_calcCHindex(*data, labels*)
labelGluer(*labels*)
labelReorder(*labels*)
_calcQuiescentFrame(*spectralData, spectralParams, contIdxs, progress=None*)
_calcDynamicFrame(*spectralData, dynamicScanNum, progress=None, delta=0*)

4.4 approach.py

__init__(*self, config, catalogName, instrumentObj*)
__getattr__(*self, name*)
cluster(*self, prepSquare, maskLine*)
__init__(*self, catalogBase*)
clustering(*self, prepCube, tlst, groups, intrinsicSquare=None*)

Detail

low temporal resolution clustering

4.5 logg.py

loggTimer(*func*)

wrapper(*args, **kwargs)

logg(tag, val=None, _time=None, _log=None)

duration_string(dur)

4.6 aia.py

delayAIA(fname, epochDev)

4.7 style.py

cbar_bounds(bounds)

_genColorPallet(n)

rainbow_cmap(nrange, discrete=False, nan=False)

cmap_discretize(cmap, N)

__init__(self, spaceInfo, deltas)

_mapGen(self, fig, pos, arr, flareContour=None, timeAxis=None, **kwargsDict)

4.8 multiline.py

4.9 products.py

__init__(self, epochObj, optLabels)

figure03(self)

figure04_template(self)

spectralEntry(self, ax, indx, color, wavelambda, extent, scores=None)

4.10 loader.py

__init__(self, data, home, fig)

_loadEventConfig(self, eventRunnerFname, args)

__init__(self, dataPath)

vispLoad(self, stokes=0)

irisLoad(self)

```

fissLoad(self, labels)

__init__(self, config1, config2)

visp2visp(self)

fiss2fiss(self)

__init__(self, dataPath, labels)

__init__(self, dataPath, stokes=0)

__init__(self, inputLst, runIndx)

loadSource(self, eventInput)

__init__(self, srcInput)

__init__(self, runnerInput)

__init__(self, fname, eventIndx, runIndx)

_load(self, fname)

```

4.11 base.py

```

_mainIntrinsic(config, prepSquare, lineIndx, intrinsicSkip=False)

_mainOptimization(prepSquare, labelLine, kLst=None, stageMax=2)

```

4.12 base.py

```

_runOptimalKSearch(dataSquare, funcLst, checkLst)

runPrep(dataSquare, norm='continuum', keepI0=None, maskLine=None, quSquare=None,
        **kwargs)

_runIntrinsic(nbins, data, edgeOverride=None)

runStart(k, data, start='max')

_runOptimization(k, sub_data, converge)

_findOptimalK(dataSquare, funcLst, criteriaLst, converge=1e-6)

_runLabelSort(dataSquare, labelLine)

```